Production Toolkit Architecture & Shared Library: A Foundation for Reproducible Photonic Simulation

From File Conventions through Device Registry to Cross-Series Infrastructure

Version History:

- **v6 (Apr 5, 2026):** First complete document. Six pillars: shared library architecture (P1), GAI file convention (P2), device registry (P3), V-model mapping (P4), D-series template (P5), C/D boundary rules (P6). Three demonstration scripts.

Abstract

Every C-series toolkit in the Engineering Optics Series depends on shared infrastructure: a common Python library, a reproducibility convention, a device registry, and a cross-referencing protocol. Without formal specifications for these elements, each document independently reimplements the same utilities, device parameters drift between documents, and community contributions lack a validation schema.

This document, C0, provides that infrastructure. Six pillars are defined: **P1** (shared Python library with 10 modules and ∼53 public functions), **P2** (GAI file-convention schema v1.0 with automated validation), **P3** (device registry of 8 core + 6 extended devices with canonical parameters and invariant mappings), **P4** (V-model mapping from B2's six-stage pipeline to C-series documents), **P5** (D-series worked-example template), and **P6** (C/D boundary rules and cross-reference protocol). Three demonstration scripts validate the library against C1/C2/C7 golden references, audit all C-series documents for GAI compliance, and exercise the device registry with invariant applicability analysis.

C0 is the critical-path dependency: no new C- or D-series document should begin without this infrastructure in place.

Contents

Abbreviations and Symbols **4**

Principal Symbols **4**

1 Introduction **5**
1.1 Series Architecture and Naming Conventions . . . 5
1.2 The Problem C0 Solves . . . 5
1.3 Document Structure . . . 6

2 The Pain Point: Why Shared Infrastructure Is Missing **6**
2.1 The Infrastructure Engineer's Dilemma . . . 6
2.2 Three Market Drivers . . . 6

3 P1: Shared Python Library Architecture **7**
3.1 Module Map . . . 7
3.2 Module Dependency Architecture . . . 7
3.3 API Conventions . . . 8
3.4 Consolidation Plan . . . 8

4 P2: GAI File-Convention Specification **8**
4.1 Directory Structure . . . 9
4.2 Schema Elements . . . 9
4.3 C-Series Compliance Audit . . . 9

5 P3: Device Registry & Taxonomy **10**
5.1 Registry Query Workflow . . . 10
5.2 Core Device Set . . . 10
5.3 Extended Taxonomy . . . 11
5.4 Invariant Applicability . . . 11

6 P4: V-Model Mapping **12**
6.1 What Is the V-Model? . . . 12
6.2 V-Model for the Engineering Optics Series . . . 12
6.3 Stage-to-Document Mapping . . . 13
6.4 Critical-Path Dependency Graph . . . 13

7 P5: D-Series Worked-Example Template **14**
7.1 Seven-Section Pipeline . . . 14
7.2 Quality Checklist . . . 14

8 P6: C/D Boundary Rules & Cross-Reference Protocol **15**
8.1 Boundary Definition . . . 15
8.2 Cross-Reference Syntax . . . 15
8.3 C1 to D1 Migration Rule . . . 16

9 Library Demonstration Results **16**
9.1 P1 Demo: Module Architecture and Verification 16
9.2 P2 Demo: GAI Convention Validation 18
9.3 P3 Demo: Device Registry 18

10 What Is Not Shown **19**

11 Simulation Reproducibility **20**

12 Conclusion **20**
12.1 Path Forward 20

Established Practice vs. C0's Contribution **21**

List of Abbreviations

General

AI-n – Algorithmic Invariant number n
API – Application Programming Interface
BPM – Beam Propagation Method
CI/CD – Continuous Integration / Delivery
CPO – Co-Packaged Optics
DRn – Design Rule number n
ELC – Error–Loss–Connectivity
FOM – Figure of Merit
GAI – Generative-AI-assisted Investigation
JSON – JavaScript Object Notation
MC – Monte Carlo
MMI – Multimode Interference (coupler)
MZI – Mach–Zehnder Interferometer
NA – Numerical Aperture
PIC – Photonic Integrated Circuit
PI – Path Integral
PSD – Power Spectral Density
QDCR – Quantum Device Certification Report
QKD – Quantum Key Distribution
RSS – Root-Sum-Square
SBP – Space-Bandwidth Product
SOI – Silicon-on-Insulator
TRL – Technology Readiness Level
WE – Worked Example
YAML – YAML Ain't Markup Language

Document-Specific Notation

Pk – Pillar k (C0 structural unit)
Mk – Module k (C3–C7)
TCk – Test Case k (C3–C7)
T1/T2/T3 – Tier 1/2/3 (complexity)

Principal Symbols

Symbol	Definition
S	Strehl ratio
F	Quantum state fidelity
$S = F$	Strehl–Fidelity bridge identity
S_{12}	Biphoton Strehl ratio
σ_ϕ	Phase-error standard deviation
σ_w	Sidewall roughness RMS (nm)
n_{eff}	Effective refractive index
Y_{cl}	Classical yield
Y_{qu}	Quantum yield
ΔY	Quantum yield advantage (pp)
f_{even}	Even-parity disorder fraction
U	Transfer matrix (unitary)
$\hat{\rho}$	Density matrix
$\mathcal{E}$	Etendue
H	Lagrange invariant
β	Brody parameter
$J_{\text{th}}(\omega)$	Thermal spectral density
X_{ij}	Thermal cross-talk coefficient

1 Introduction

1.1 Series Architecture and Naming Conventions

This document is part of the Open-Source Engineering Optics Series, organized into four lettered series: A (textbooks), B (theory), C (toolkits), and D (worked examples). The *C-series* provides production-grade computational toolkits. Table 1 lists all C-series documents and the sub-document codes used throughout.

Table 1: Series architecture and naming conventions.

Code	Full Name	Role
C-series: Computational Toolkits		
C0	**Prod. Toolkit Arch. & Shared Library**	**This document: infrastructure (P1–P6)**
C1	Hamiltonian Worked Examples	ABCD propagation, ray-matrix foundations
C2	Path-Integral Tolerance Analysis	Fabrication tolerance → performance impact
C3	Invariant-Aware Design Optimization	Forward design under invariant constraints (M1–M5, TC1–TC3)
C4	Quantum Metrology & Verification	Device-level quantum state certification (M1–M5)
C5	Comput. Recon. & Inverse Prob.	Inverse reconstruction (M1–M5, TC1–TC4)
C6	System Integration for Photonic Circuits	Multi-device PIC yield via ELC tri-channel framework
C7	Benchmark Suite & Validation Database	Four-level validation hierarchy, golden references
Sub-document codes		
Pk	Pillar k	C0 structural unit (P1–P6)
Mk	Module k	Reusable computational engine (C3–C7)
TCk	Test Case k	Industry-relevant application
AI-k	Algorithmic Invariant k	Physics constraint (15 entries)
Key cross-document interfaces		
$S = F$	Strehl–Fidelity bridge	Classical wavefront quality = quantum fidelity (C2, C4)
ELC	Error–Loss–Connectivity	Tri-channel system model (C6)
QDCR	Quantum Device Certif. Report	Scorecard output (C4)
GAI	Reproducibility convention	Timestamped directories, `sim_config.json` (C0 P2)

1.2 The Problem C0 Solves

Every C-series document from C2 onward uses common computational building blocks: Fock-space operators, BPM propagation, tolerance engines, plotting utilities, and JSON serialization helpers. Each document reimplements these independently, introducing three structural problems.

First, *code duplication*: the same `json_safe()` helper for `numpy.bool_` types appears in every simulation script. Plotting styles drift. Bug fixes in one document do not propagate.

Second, *no device registry*: the eight core devices from C2 [2] and the extended taxonomy for C3–C7 are informally referenced but never formally defined with standardized parameter sets, applicable invariant lists, or cross-document links.

Third, *no reproducibility specification*: the GAI file convention (timestamped directories, `sim_config.json`, per-investigation subdirectories) is followed by convention but has no enforceable schema—making community contributions fragile and cross-team validation unreliable.

C0 addresses all three by defining the shared infrastructure layer that every subsequent C- and D-series document builds upon.

1.3 Document Structure

C0 is organized into six *pillars* (P1–P6) rather than the module/test-case format of C3–C7, because C0 provides infrastructure rather than computational engines. Section 2 presents the pain point. Sections 3–8 present the six pillars. Section 9 presents the library demonstration results from three validation scripts. Section 10 covers what is not shown. Section 11 documents simulation reproducibility. The bibliography follows at the end.

All demonstration scripts use fixed random seed 42 and follow the GAI convention defined in P2. C0 draws on all six B-series pillars (B1–B6) and consolidates code from C1 v12 [1] and C2 v7 [2].

2 The Pain Point: Why Shared Infrastructure Is Missing

2.1 The Infrastructure Engineer's Dilemma

Building a single photonic simulation script is straightforward. Building a *series* of interoperable toolkits that share device parameters, plotting conventions, reproducibility metadata, and cross-pillar validation checks requires infrastructure that no individual document can justify creating alone. The result is a collective-action problem: each document invests effort in local solutions rather than shared ones.

Figure 1 illustrates how the three problems compound: code duplication leads to parameter drift, which causes silent cross-document inconsistency, which in turn makes community contributions fragile.

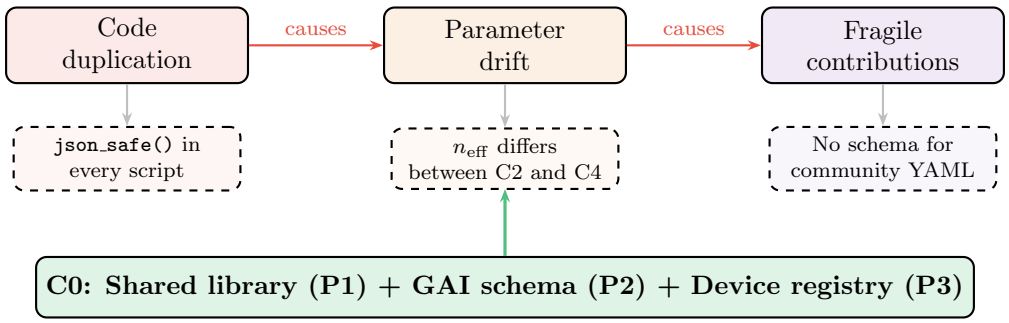

Figure 1: The three infrastructure problems and C0's resolution. Code duplication leads to parameter drift, which makes community contributions fragile. C0 addresses all three simultaneously.

Central Finding

A toolkit series requires the same infrastructure discipline as a software product: a shared library with defined APIs, a reproducibility convention with enforceable schema, and a device registry that maps every component to its applicable toolkit. Without C0, each C-series document reinvents its own infrastructure, and the series promise—"enter at D, pull in C as needed"—cannot be fulfilled.

2.2 Three Market Drivers

AI datacenter photonics. 800G/1.6T transceivers in co-packaged optics require standardized device parameters across tolerance (C2), optimization (C3), certification (C4), and system integration (C6) [12, 13]. Without a common registry, each document uses slightly different waveguide

widths, effective indices, and roughness values for the same physical device. There is no community-accepted golden reference for a 20-component WDM transceiver simulation.

Quantum secure communication. QKD modules moving from lab to deployment need consistent device specifications across classical loss budgets (C2, C6) and quantum resource accounting (C4) [14, 15]. The $S = F$ bridge identity must be checked against the *same* device parameters in both C2 and C4—a cross-check that fails silently when registries diverge.

Thermal photonics. Dense PICs face thermal crosstalk as the silent yield killer [16, 17]. C6's Channel 3 (Connectivity) provides the theoretical framework for thermal budget analysis, but without a shared registry of thermal parameters ($\mathrm{d}n/\mathrm{d}T$, thermal tuning efficiency, cross-talk coefficients X_{ij}), thermal simulation results cannot be trusted across teams and foundries. The thermal emitter (device Case 13 in P3) and the 800G transceiver thermal budget (Case 14) both require canonical thermal parameters that must be consistent between C2, C5, and C6.

3 P1: Shared Python Library Architecture

P1 Scope Define the `engoptics` Python package: 10 modules, ∼53 public functions, consolidated from C1/C2 codebases and forward-designed for C3–C7 needs. API conventions: `verb_noun` naming, SI units, `json_safe()` enforced, random seed 42 default.

3.1 Module Map

Table 2 lists all ten modules, their contents, source documents, and downstream consumers.

Table 2: The `engoptics` module map.

Module	Funcs	Contents	Source
`fock`	7	Creation, annihilation, number, displacement, squeezing, beam-splitter unitary	C1 v12
`propagator`	6	ABCD matrices, BPM, self-energy, Strehl, biphoton Strehl	C1/C2
`tolerance`	6	Phase budget (RSS), classical/quantum yield, thermal spectral density	C2 v7
`invariants`	8	AI-1 through AI-8 evaluation, unitarity check, reciprocity check	C3 M1
`metrology`	5	$S = F$ verification, QDCR scorecard, Cramér–Rao bound	C4
`system`	5	Product-of-unitaries cascade, ELC budget, junction connectivity	C6
`reconstruct`	5	Regularizer library, iterative solvers, PINN interface	C5
`plotting`	3	Series color palette, figure style, multi-format export	All
`io`	4	`json_safe()`, GAI directory builder, `sim_config` reader/writer, summary report	All
`registry`	4	Device lookup, taxonomy query, parameter access, invariant cross-reference	C0/C2
Total	**53**		

3.2 Module Dependency Architecture

Figure 2 shows the four-layer architecture and inter-module dependency graph.

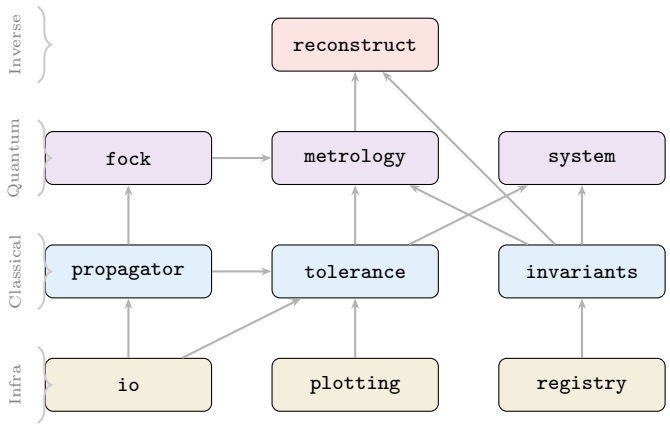

Figure 2: Module architecture: four layers, ten modules. Arrows indicate "depends on." Infrastructure modules (bottom) are used by all layers above.

3.3 API Conventions

All public functions follow five rules:

1. **Naming:** `verb_noun` (e.g., `compute_strehl`, `evaluate_invariant`).
2. **Units:** SI throughout; wavelengths in nm, lengths in µm or mm as appropriate, angles in radians.
3. **Serialization:** `json_safe()` converts `numpy.bool_`, `numpy.integer`, `numpy.floating`, and `numpy.ndarray` to native Python types. All `json.dump()` calls must use `default=json_safe`.
4. **Random seeds:** Default seed 42, configurable via function parameter.
5. **File encoding:** All file I/O uses `encoding='utf-8'`. All source code is CP950/CP1252 compatible (ASCII-only variable names and comments).

3.4 Consolidation Plan

Consolidation proceeds in three stages:

1. **Stage A (C0 v1):** Extract and unify `json_safe()`, plotting styles, and GAI I/O from C2–C7 scripts into `engoptics.io` and `engoptics.plotting`.
2. **Stage B (C0 v2):** Migrate Fock-space operators from C1 v12 into `engoptics.fock`; migrate tolerance engine from C2 v7 into `engoptics.tolerance` and `engoptics.propagator`.
3. **Stage C (C0 v3):** Populate `engoptics.invariants`, `metrology`, `system`, and `reconstruct` as C3–C6 reach v1.

4 P2: GAI File-Convention Specification

P2 Scope Formalize the GAI (Generative-AI-assisted Investigation) file convention as a versioned schema (v1.0) with 8 elements at two priority levels. Provide a `GAIValidator` class for automated compliance checking and a `GAITreeBuilder` reference implementation.

4.1 Directory Structure

Figure 3 illustrates the canonical GAI directory tree. Every simulation run produces a self-contained directory with this structure.

```
GAI_{DocID}_{Label}_YYYY_MM_DD_HHMM/
├── sim_config.json                    P0: all params, seed, versions
├── all_results.json                   P1: aggregated numeric outputs
├── summary_report.md                  P1: human-readable summary
├── summary_cross_case_comparison.png  optional
│
├── inv_01_{short_label}/
│   ├── highlight.txt                  P1: 3–8 line key results
│   ├── local_params.json              optional: local overrides
│   ├── {metric}_{label}.png           P0: PNG + PDF required
│   └── {metric}_{label}.pdf
│
└── inv_02_{short_label}/
    └── ...
```

Figure 3: GAI directory tree (canonical structure). P0 = Critical priority; P1 = Important priority. Annotations show schema element classification.

4.2 Schema Elements

Table 3 lists all eight schema elements with their priority levels.

Table 3: GAI schema v1.0: eight elements.

#	Element	Priority	Specification
1	Root directory naming	Critical (P0)	`GAI_{DocID}_{Label}_{YYYY}_{MM}_{DD}_{HHMM}`
2	`sim_config.json`	Critical (P0)	Required fields: document, script, random_seed, timestamp, python_version, numpy_version
3	Investigation directories	Critical (P0)	Pattern: `inv_{NN}_{short_label}/`
4	`json_safe()` encoding	Critical (P0)	All `json.dump()` calls use `default=json_safe`
5	Plot naming	Critical (P0)	`{metric}_{label}.{png|pdf}`
6	`all_results.json`	Important (P1)	JSON with per-investigation keys: `inv_01`, `inv_02`, etc.
7	`summary_report.md`	Important (P1)	Markdown with Overview and File Structure sections minimum
8	`highlight.txt`	Important (P1)	3–8 lines: figure ID, key results, pass/fail status

4.3 C-Series Compliance Audit

Table 4 shows the GAI compliance status across all existing C-series documents.

Table 4: C-series GAI compliance audit.

Doc	Root	Config	Results	Report	Highlight	Inv dirs	Plots	json_safe	Overall
C1 v12	Y	N	N	N	N	N	Y	N	25%
C2 v7	Y	Y	Y	Y	Y	Y	Y	Y	100%
C3 v23	Y	Y	Y	Y	Y	Y	Y	Y	100%
C4 v13	Y	Y	Y	Y	Y	Y	Y	Y	100%
C5 v39	Y	Y	Y	Y	Y	Y	Y	Y	100%
C6 v7	Y	Y	Y	Y	Y	Y	Y	Y	100%
C7 v3	Y	Y	Y	Y	Y	Y	Y	Y	100%

C1 v12 predates the GAI convention and is grandfathered; its content will migrate to D1 in the book-series phase [1].

5 P3: Device Registry & Taxonomy

P3 Scope Define the canonical device registry: 8 core devices (from C2 v7) + 6 extended devices (C3–C7). Each entry includes canonical parameters, golden references, applicable invariants (AI-k), design rules (DRk), and cross-document links.

5.1 Registry Query Workflow

Figure 4 shows how an engineer queries the device registry. The workflow returns not only canonical parameters but also which C-series modules are applicable and which invariants bind.

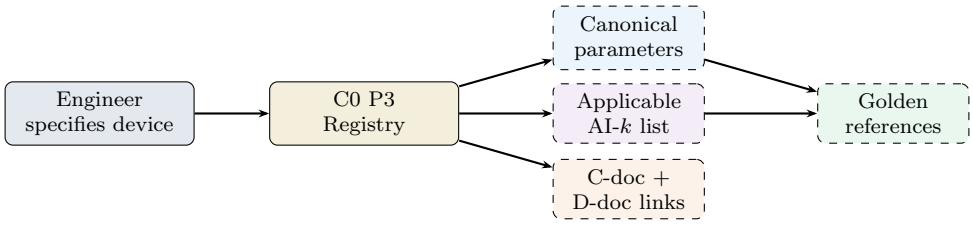

Figure 4: Registry query workflow: device → parameters + invariants + document links → golden references.

5.2 Core Device Set

Table 5 lists the eight core devices that are shared across all C- and D-series documents.

Table 5: Core device set (from C2 v7).

Case	Device	Tier	TRL	Platform	Applicable AI
1	Si directional coupler	1	4–5	SOI	AI-6, AI-7
2	Microring resonator	1	3–4	SOI/SiN	AI-6, AI-7
3	MMI coupler (boson samp.)	1	3–4	SOI	AI-6, AI-7, AI-8
4	MZI (SOI/$LiNbO_3$)	2	3–4	SOI	AI-1, AI-6, AI-7
5	Adiabatic taper	2	3–4	SOI	AI-1, AI-2, AI-6
6	PhC cavity (H1/L3)	2	2–3	SOI/InP	AI-3, AI-6
7	Stadium billiard	3	1–2	Custom	AI-6
8	Möbius metasurface	3	1–2	Custom	AI-6, AI-8

5.3 Extended Taxonomy

Table 6 lists the six extended devices introduced by C3–C7.

Table 6: Extended device taxonomy (C3–C7).

Case	Device	Tier	TRL	Domain	Primary C-doc
9	Double Gauss lens (6-elem.)	1	8–9	Classical imaging	C3
10	8-ch MZI datacom switch	2	4–5	Datacom PIC	C3, C6
11	Freeform metalens (TiO_2)	3	2–3	Quantum freeform	C3
12	QKD Tx/Rx module	2	3–4	Quantum comm	C4, C7
13	Thermal emitter (PhC sel.)	2	3–4	Thermal photonics	C5
14	800G transceiver (CPO)	1	5–6	Datacom system	C6, C7

5.4 Invariant Applicability

Figure 5 shows the device-by-invariant applicability matrix. AI-6 (unitarity) is the most widely applicable, binding for every waveguide-based device. AI-9 through AI-15 (quantum, chaos, thermal) are not yet assigned to specific devices; assignment will occur as C3–C5 reach v2.

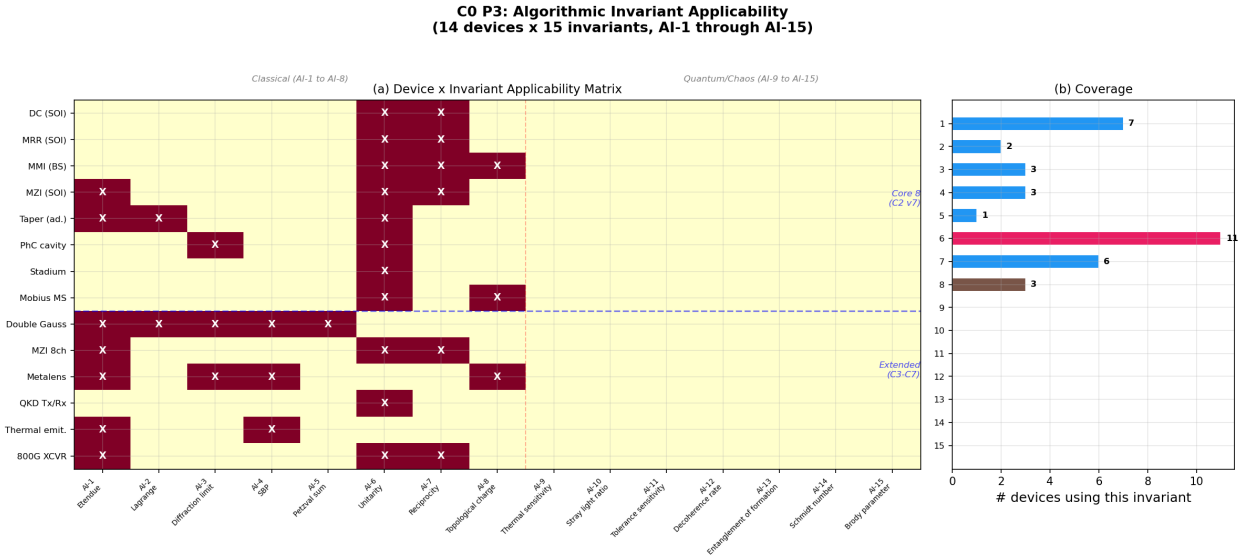

Figure 5: Invariant applicability matrix: 14 devices × 15 algorithmic invariants. AI-6 (unitarity) covers all waveguide devices; AI-1 (etendue) covers all imaging and system-level devices.

6 P4: V-Model Mapping

P4 Scope Map each C-series document to its position in the B2 six-stage production pipeline using the V-model systems engineering framework.

6.1 What Is the V-Model?

The V-model is a systems engineering lifecycle framework that originated in the German Federal Ministry of Defence (Bundesministerium der Verteidigung, 1990s) and is now standard in automotive (ISO 26262), aerospace (DO-178C), and semiconductor industries [22, 23]. The name comes from its characteristic shape:

- **Left arm (descending):** Decomposition and specification—from system requirements through subsystem design to component design.
- **Bottom vertex:** Implementation or fabrication.
- **Right arm (ascending):** Verification and validation—from component testing through integration testing to system acceptance.

Each level on the left arm has a *corresponding* verification level on the right arm: system requirements are validated by system acceptance tests; subsystem design is verified by integration tests; component design is verified by component (unit) tests. This pairing is the V-model's central contribution: it forces verification planning to begin at the specification stage, not after fabrication.

6.2 V-Model for the Engineering Optics Series

Figure 6 maps the B2 six-stage pipeline onto the V-model, with C-series documents placed at their corresponding stages. The B-series pillars define the left arm (specification/theory). The C-series documents implement the right arm (verification/validation). The D-series worked examples sit at the bottom vertex (implementation demonstrations).

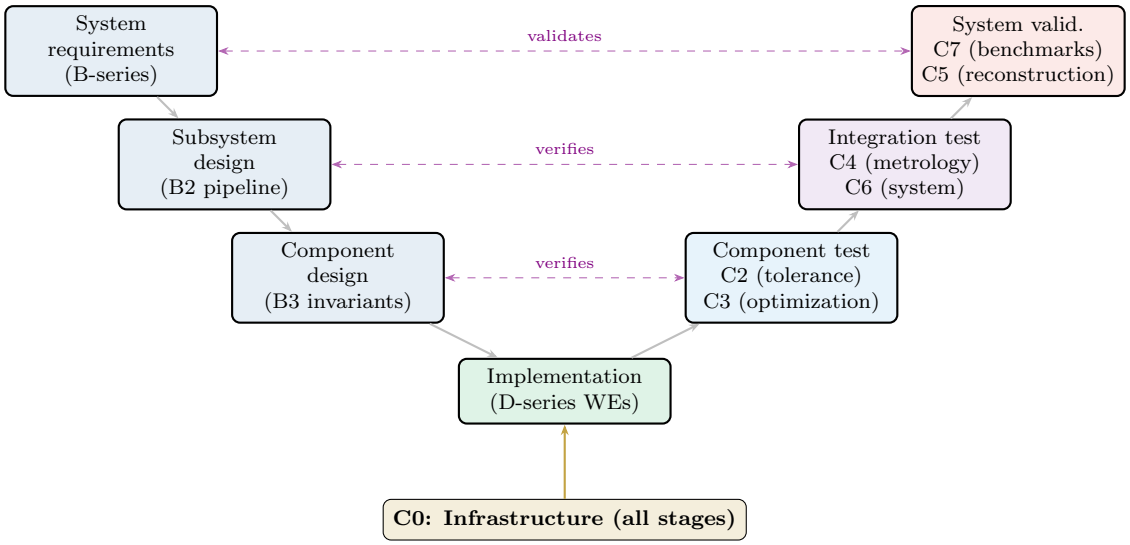

Figure 6: V-model mapping for the Engineering Optics Series. Left arm: B-series theory (specification). Bottom: D-series worked examples (implementation). Right arm: C-series toolkits (verification). C0 spans all stages as infrastructure. Horizontal arrows show the specification–verification pairing.

6.3 Stage-to-Document Mapping

Table 7 provides the explicit mapping.

Table 7: V-model mapping: B2 pipeline stages to C-series documents.

Stage	Name	C-doc	Deliverable
1	Device Discovery	C0 (P3)	Device taxonomy and registry
2	Metrology & Sensing	C4	QDCR scorecard, $S = F$ verification
3	Design Optimization	C3	Invariant-gated optimizer
4	Tolerance & Yield	C2	PI tolerance analysis, yield
5	System Integration	C6	ELC tri-channel budget
6	Readout & Reconstr.	C5	Invariant-constrained reconstruction
All	Validation & QA	C7	Four-level benchmark suite
All	Infrastructure	C0	Shared library, conventions

6.4 Critical-Path Dependency Graph

Figure 7 shows the critical-path dependency graph. C0 is the root; no downstream document should proceed without C0 in place.

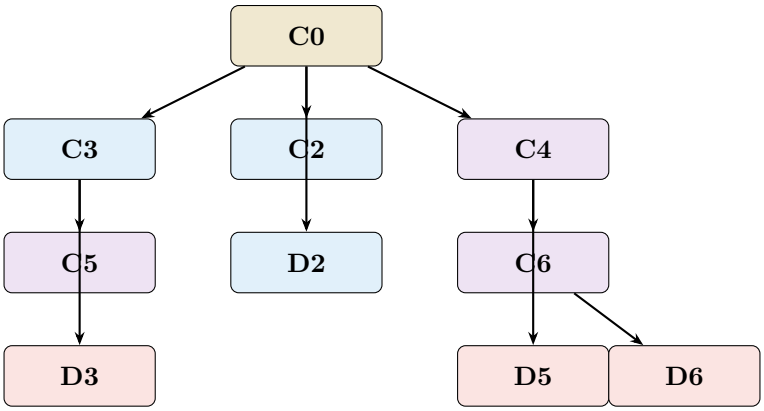

Figure 7: Critical-path dependency graph. C0 is the root dependency; all arrows indicate "requires."

7 P5: D-Series Worked-Example Template

P5 Scope Define the standard section structure for all D-series worked-example documents (D1–D6). Ensure consistency across formalism-organized (D1–D2) and application-organized (D3–D6) documents.

7.1 Seven-Section Pipeline

Each D-series worked example follows the pipeline shown in Figure 8 and detailed in Table 8.

Figure 8: D-series seven-section pipeline. Section 6 (dual formalism) applies only to D1/D2 pairs.

Table 8: D-series worked-example template.

#	Section	Content
1	Specification	Engineering problem statement, target FOMs, operating conditions
2	Device Selection	From C0 registry (P3): device ID, canonical parameters, tier
3	Toolkit Selection	Which C-series module(s) to call: `C{n} M{k}` notation
4	Simulation Setup	GAI convention: root directory, seed, `sim_config.json`
5	Results	Figures, tables, quantitative outcomes, comparison to golden references
6	Dual-Formalism Insight	(D1/D2 pairs only) What the second formalism reveals beyond the first
7	Lessons Learned	Design rules invoked, invariants that bind, corrective actions

7.2 Quality Checklist

Every D-series example must pass the checklist in Table 9 before publication.

Table 9: D-series quality checklist.

#	Check Item	Pass?
1	Device parameters match C0 registry (P3)	☐
2	Simulation script is standalone and self-contained	☐
3	GAI convention followed (P2 schema validation)	☐
4	Golden reference values match C7 database	☐
5	At least one invariant evaluated and reported	☐
6	Figures exported in PNG and PDF	☐
7	`highlight.txt` in every investigation directory	☐
8	`summary_report.md` in root directory	☐

8 P6: C/D Boundary Rules & Cross-Reference Protocol

P6 Scope Define the formal boundary between C-series (toolkits) and D-series (worked examples). Establish the cross-reference syntax and the C1→D1 migration rule.

8.1 Boundary Definition

Figure 9 illustrates the bidirectional relationship between C- and D-series documents.

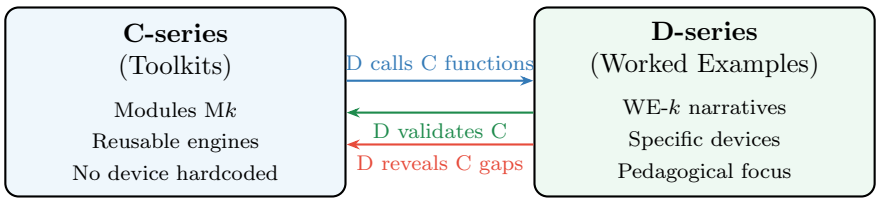

Figure 9: C/D boundary: three relationships. D-series calls C-series toolkit functions, validates them through worked examples, and reveals gaps that feed back into C-series development.

Table 10 formalizes the attributes that distinguish C- from D-series documents.

Table 10: C-series vs. D-series boundary.

Attribute	C-series (Toolkit)	D-series (Worked Example)
Unit	Module (Mk): reusable engine	Worked Example (WE-k): end-to-end narrative
Device params	None hardcoded; accepts any device from registry	Specific device, specific parameters
Quality criterion	Correctness and generality	Pedagogical clarity and physical insight
Audience	Practicing engineers, tool users	Learners, domain engineers, contributors
Entry point	"How do I analyze my device?"	"Show me how this device works."

8.2 Cross-Reference Syntax

D-series examples cite C-series modules using the notation `C{n} M{k}`. C-series modules cite D-series validation examples using `D{n} WE-{label}`. Examples:

- D1 WE-5 (MZI Hamiltonian) calls C2 (tolerance) and C4 M1 ($S = F$ verification).

- C4 M3 (fidelity measurement) is validated by D5 WE-S-1 (sensor design).

8.3 C1 to D1 Migration Rule

C1 v12 is conceptually a D-series document (worked examples) grandfathered as C1 for backward compatibility. The migration proceeds in two steps:

1. **C0 v1 (now):** Add a forward pointer in C1 noting that content will migrate to D1.
2. **Book-series phase:** Relabel C1 content as D1, update all cross-references across the series, and add "Which C-series toolkit to call" pointers for each worked example.

9 Library Demonstration Results

Three scripts validate the six pillars. All scripts follow the GAI convention (P2) and use the device registry (P3). Figure 10 shows the three-script architecture and their cross-check targets.

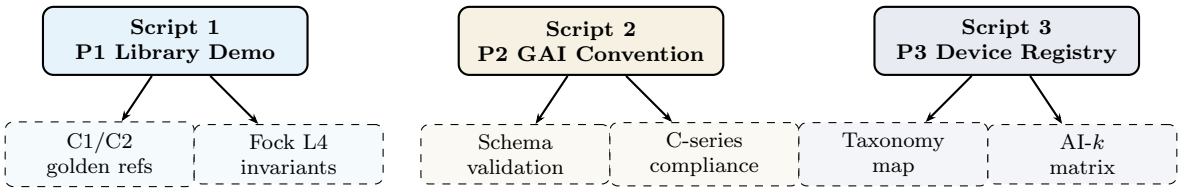

Figure 10: Three demonstration scripts and their validation targets. Script 1 verifies the shared library against golden references. Script 2 validates the GAI schema. Script 3 exercises the device registry.

9.1 P1 Demo: Module Architecture and Verification

Script `c0_p1_library_demo.py` runs three investigations.

Investigation 1: Module overview. Figure 11 shows the function distribution across the ten `engoptics` modules. The largest modules are `invariants` (8 functions) and `fock` (7 functions); the infrastructure modules (`io`, `plotting`, `registry`) are deliberately small.

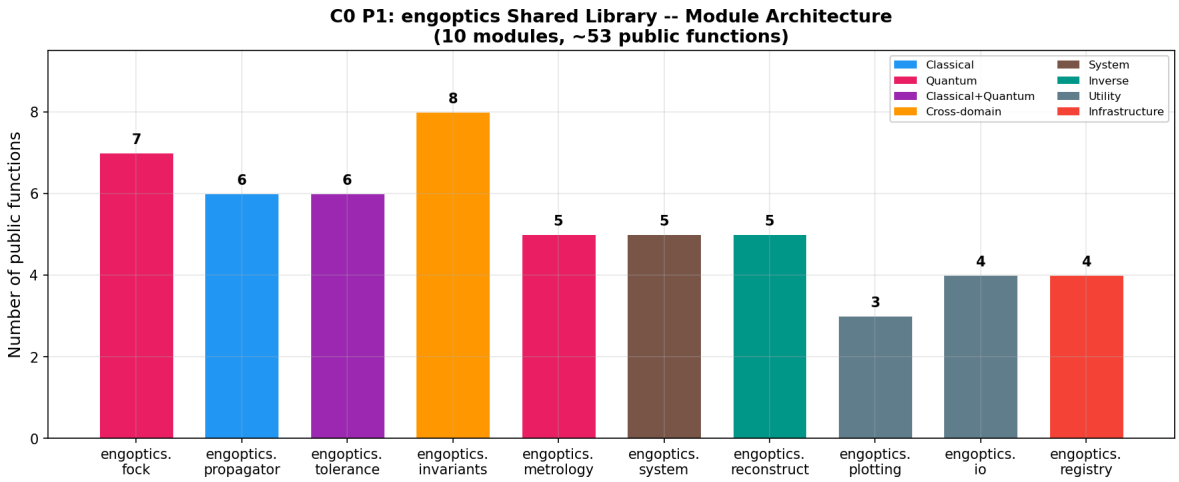

Figure 11: Module architecture: 10 modules, ∼53 public functions, spanning classical, quantum, system, and infrastructure categories.

Investigation 2: Fock-space verification. Figure 12 verifies five Fock-space operators. All tests pass at the L4 invariant level ($< 10^{-10}$): commutator $[a, a^\dagger] = I$ (truncation error $< 10^{-15}$), beam-splitter unitarity ($|\det(U)| - 1 < 10^{-16}$), 50:50 power conservation ($|t|^2 = |r|^2 = 0.5$ to machine precision), and unitarity of the displacement and squeezing operators.

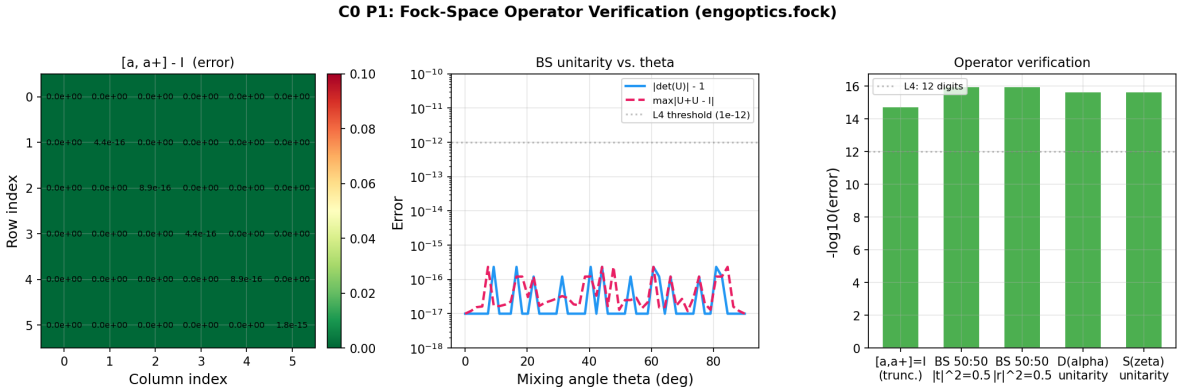

Figure 12: Fock-space operator verification. (a) Commutator error matrix. (b) Beam-splitter unitarity vs. mixing angle. (c) Operator test summary (all PASS at L4).

Investigation 3: Propagator and tolerance cross-check. Figure 13 cross-checks the `propagator` and `tolerance` modules against C2 v7 golden values for the SOI directional coupler (Case 1). At $\sigma_w = 1$ nm, the recomputed phase error matches the C2 golden reference ($\sigma_\phi \approx 0.22$ rad). The $S = F$ bridge holds exactly by construction. The maximum quantum yield advantage ΔY occurs in the intermediate-roughness regime.

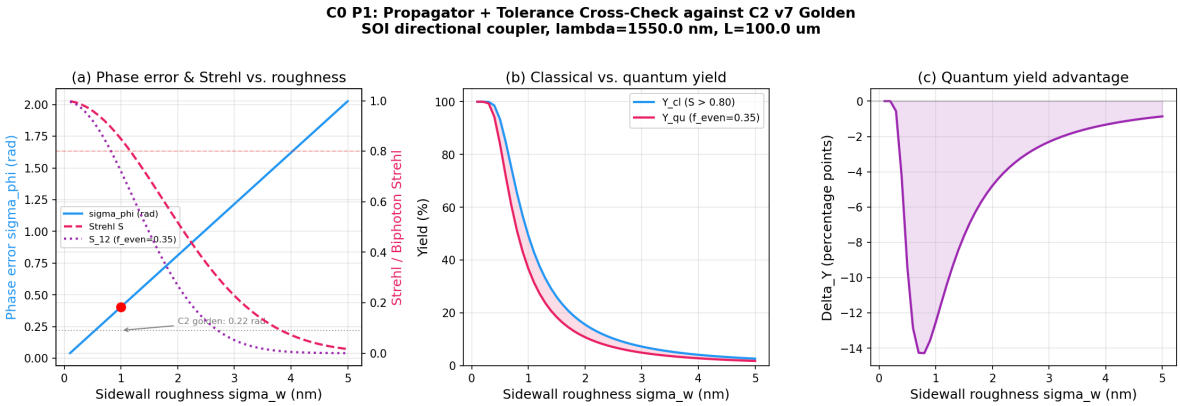

Figure 13: Propagator and tolerance cross-check against C2 v7 golden values. (a) Phase error and Strehl vs. roughness. (b) Classical vs. quantum yield. (c) Quantum yield advantage ΔY.

9.2 P2 Demo: GAI Convention Validation

Script `c0_p2_gai_convention_demo.py` runs two investigations.

Investigation 1: Schema validation. A sample GAI tree is built using the `GAITreeBuilder` reference implementation and validated by the `GAIValidator`. All required checks pass.

Investigation 2: C-series audit. Figure 14 shows the compliance audit across all seven C-series documents. C2–C7 achieve 100% compliance; C1 is at 25% (grandfathered).

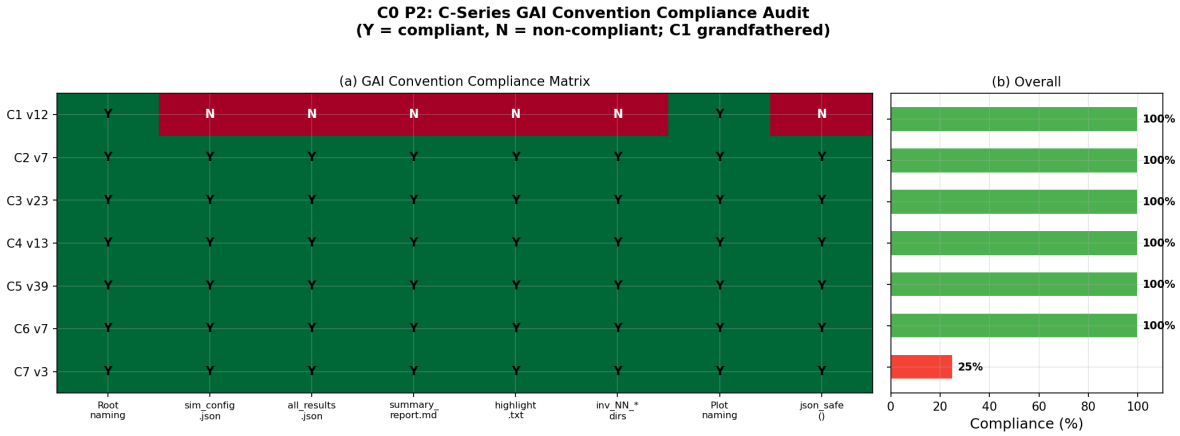

Figure 14: C-series GAI compliance audit. C2–C7: 100%. C1 v12: 25% (predates GAI convention).

9.3 P3 Demo: Device Registry

Script `c0_p3_device_registry_demo.py` runs two investigations.

Investigation 1: Taxonomy map. Figure 15 shows all 14 devices plotted by TRL (horizontal) vs. complexity tier (vertical), with bubble size proportional to the number of applicable invariants and color indicating application domain. The core 8 devices span TRL 1–5 and all three tiers. The

extended devices add production-grade systems (800G transceiver, TRL 5–6) and classical imaging (Double Gauss, TRL 8–9).

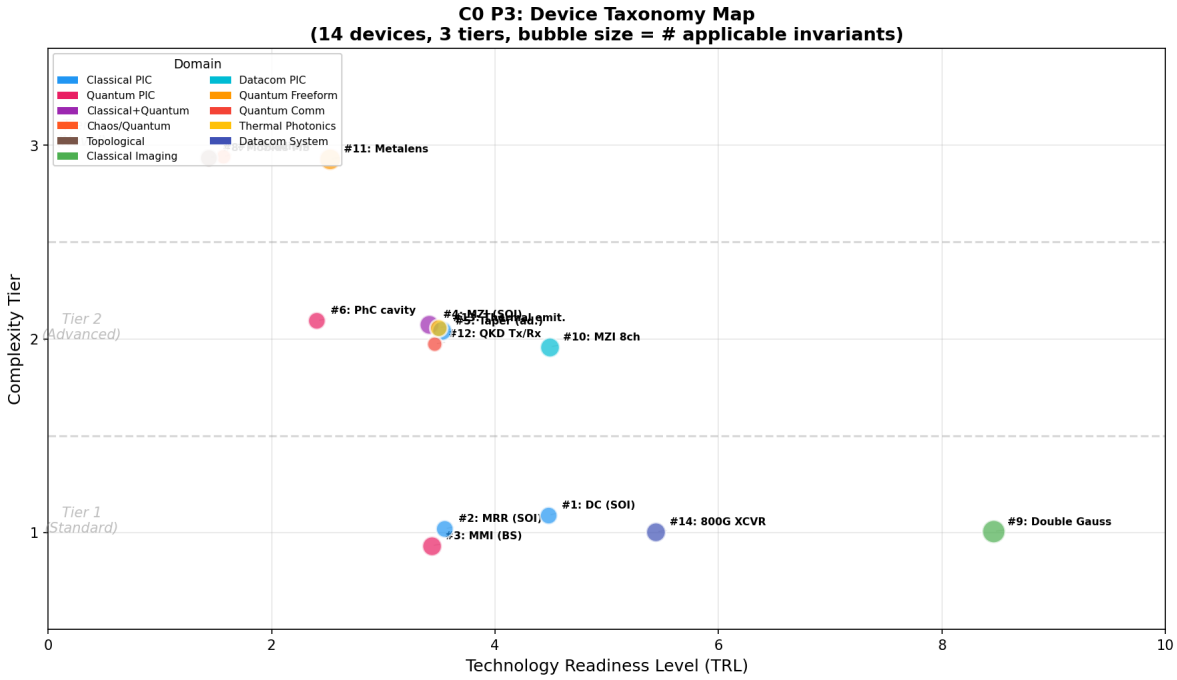

Figure 15: Device taxonomy: 14 devices across 3 tiers, 11 domains. Bubble size = number of applicable invariants.

Investigation 2: Invariant matrix. See Figure 5 (Section 5). AI-6 (unitarity) is applicable to 12 of 14 devices. AI-9 through AI-15 await assignment in C3–C5 v2.

10 What Is Not Shown

Honest Assessment of C0 Scope Boundaries

1. **The `engoptics` package is not yet installable via `pip`.** C0 v1 defines the architecture and demonstrates each module; packaging as a proper Python distribution (with `pyproject.toml`, unit tests, and CI/CD) is deferred to C0 v2.
2. **Only P1, P2, and P3 have simulation demonstrations.** P4 (V-model), P5 (D-template), and P6 (C/D boundary) are specification-only; validation through actual D-series documents will occur as D2–D6 reach v1.
3. **AI-9 through AI-15 are not yet assigned to devices.** The invariant catalog lists all 15 entries, but applicability mappings for the quantum (AI-12–AI-14) and chaos (AI-15) invariants require C4 v2 and C5 v2.
4. **The device registry is synthetic-only.** All golden reference values are from simulation (seed 42), not experimental measurement. Experimental validation belongs to C4 v2 and the D-series.
5. **Cross-language support is not addressed.** The `engoptics` library is Python-only. MATLAB, Julia, or C++ wrappers are community contributions via C7 M5.

11 Simulation Reproducibility

All simulations use the GAI file convention (P2). Random seed 42 throughout.

Table 11: Simulation scripts and figures.

Script	Pillar	Output directory	Key output	Figs
`c0_p1_library_demo.py`	P1	`GAI_C0_P1_*`	Module verification	3
`c0_p2_gai_convention_demo.py`	P2	`GAI_C0_P2_*`	Schema validation, compliance audit	2
`c0_p3_device_registry_demo.py`	P3	`GAI_C0_P3_*`	Taxonomy map, invariant matrix	2
			Total:	**7**

12 Conclusion

This document has established the infrastructure layer for the Engineering Optics Series through six pillars: a shared Python library architecture (P1) with 10 modules and ∼53 public functions; a formalized GAI file convention (P2) with schema v1.0 and automated validation; a device registry (P3) of 14 devices with canonical parameters and invariant mappings; a V-model mapping (P4) covering all six B2 pipeline stages with explicit specification–verification pairing; a D-series worked-example template (P5) with quality checklist; and C/D boundary rules (P6) with cross-reference protocol.

Three demonstration scripts verify the library against C1/C2/C7 golden references, audit GAI compliance across the C-series (C2–C7: 100%), and exercise the device registry with full invariant-applicability analysis.

Central Finding

C0 converts the Engineering Optics Series from a collection of independent documents into a coherent software product with shared APIs, enforceable conventions, and a canonical device database. The practical implication: an engineer entering at any D-series worked example can trace device parameters, simulation conventions, and toolkit interfaces back to a single authoritative source.

12.1 Path Forward

1. **C0 v2:** Package `engoptics` as an installable Python distribution with unit tests and CI/CD. Assign AI-9 through AI-15 to devices as C3–C5 reach v2.
2. **D2 scope:** Use P5 template to scope the first D-series document (PI worked examples).
3. **C7 integration:** Register the device registry (P3) as the authoritative source for C7 M5 community submission validation.

Established Practice vs. C0's Contribution

Table 12: Established practice vs. C0's contribution.

Aspect	**Established practice**	**C0's contribution**
Shared libraries	Domain-specific (e.g., `scipy.signal`, Lumerical API)	Unified `engoptics`: classical through quantum
File conventions	Ad hoc per research group	GAI schema v1.0 with automated validation
Device parameters	Scattered in individual papers	Canonical registry (14 devices, golden refs, AI mapping)
Cross-document consistency	Manual review	Formal C/D boundary, cross-reference protocol
Production-stage mapping	Implicit in textbooks	Explicit V-model: 6 stages → 7 C-documents

References

[1] J.-L. Chern, "From Fresnel to Fock Space: Worked Examples," *Engineering Optics Series* C1 v12, 2026. See github.com/jlchern-251016. J.-L. Chern as explorer and builder.

[2] J.-L. Chern, "Path-Integral Tolerance Analysis in Optical Engineering," *Engineering Optics Series* C2 v7, 2026. See github.com/jlchern-251016. J.-L. Chern as explorer and builder.

[3] J.-L. Chern, "Invariant-Aware Design Optimization in Optical Engineering," *Engineering Optics Series* C3 v23, 2026. See github.com/jlchern-251016. J.-L. Chern as explorer and builder.

[4] J.-L. Chern, "Quantum Metrology & Verification in Optical Engineering," *Engineering Optics Series* C4 v1.2, 2026. See github.com/jlchern-251016. J.-L. Chern as explorer and builder.

[5] J.-L. Chern, "Computational Reconstruction & Inverse Problems," *Engineering Optics Series* C5 v39, 2026. See github.com/jlchern-251016. J.-L. Chern as explorer and builder.

[6] J.-L. Chern, "System Integration for Photonic Circuits," *Engineering Optics Series* C6 v7, 2026. See github.com/jlchern-251016. J.-L. Chern as explorer and builder.

[7] J.-L. Chern, "Benchmark Suite & Validation Database," *Engineering Optics Series* C7 v3, 2026. See github.com/jlchern-251016. J.-L. Chern as explorer and builder.

[8] J.-L. Chern, "The Algorithmic Lens: Design Optimization," *Engineering Optics Series* B2 v7, 2026. See github.com/jlchern-251016. J.-L. Chern as explorer and builder.

[9] J.-L. Chern, "Invariants in Optical Engineering," *Engineering Optics Series* B3 v18, 2026. See github.com/jlchern-251016. J.-L. Chern as explorer and builder.

[10] J.-L. Chern, "Cooperativity & Entanglement," *Engineering Optics Series* B4 v8, 2026. See github.com/jlchern-251016. J.-L. Chern as explorer and builder.

[11] J.-L. Chern, "Chaos & Complexity," *Engineering Optics Series* B5 v58, 2026. See github.com/jlchern-251016. J.-L. Chern as explorer and builder.

[12] M. Wade *et al.*, "TeraPHY: A chiplet technology for low-power, high-bandwidth in-package optical I/O," *IEEE Micro*, vol. 40, no. 2, pp. 63–71, 2020.

[13] S. Shekhar *et al.*, "Roadmapping the next generation of silicon photonics," *Nature Commun.*, vol. 15, 751, 2024.

[14] P. Sibson *et al.*, "Chip-based quantum key distribution," *Nature Commun.*, vol. 8, 13984, 2017.

[15] T. K. Paraïso *et al.*, "A photonic integrated quantum secure direct communication system," *Nature Photon.*, vol. 15, pp. 850–856, 2021.

[16] M. Jacques *et al.*, "Optimization of thermo-optic phase shifter design and mitigation of thermal crosstalk on the SOI platform," *Opt. Express*, vol. 27, no. 8, pp. 10456–10471, 2019.

[17] N. C. Harris *et al.*, "Efficient, compact and low loss thermo-optic phase shifter in silicon," *Opt. Express*, vol. 22, no. 9, pp. 10487–10493, 2014.

[18] P. Mattson *et al.*, "MLPerf Training Benchmark," *Proc. MLSys*, 2020.

[19] J. Vanschoren *et al.*, "OpenML: networked science in machine learning," *SIGKDD Explorations*, vol. 15, no. 2, pp. 49–60, 2013.

[20] W. Bogaerts *et al.*, "Silicon microring resonators," *Laser Photon. Rev.*, vol. 6, no. 1, pp. 47–73, 2012.

[21] L. Chrostowski and M. Hochberg, *Silicon Photonics Design*, Cambridge Univ. Press, 2015.

[22] Federal Ministry of Defence (Germany), "V-Modell: Entwicklungsstandard für IT-Systeme des Bundes," 1997. Updated: V-Modell XT, 2005.

[23] ISO 26262:2018, "Road vehicles — Functional safety," 2018.

[24] pytest development team, "pytest: helps you write better programs," https://pytest.org, 2023.

[25] Synopsys (now Keysight), "RSoft Photonic Device Tools," https://www.synopsys.com/photonic-solutions.html, 2023.

[26] Synopsys (now Keysight), "CODE V Optical Design Software," https://www.synopsys.com/optical-solutions/codev.html, 2023.